

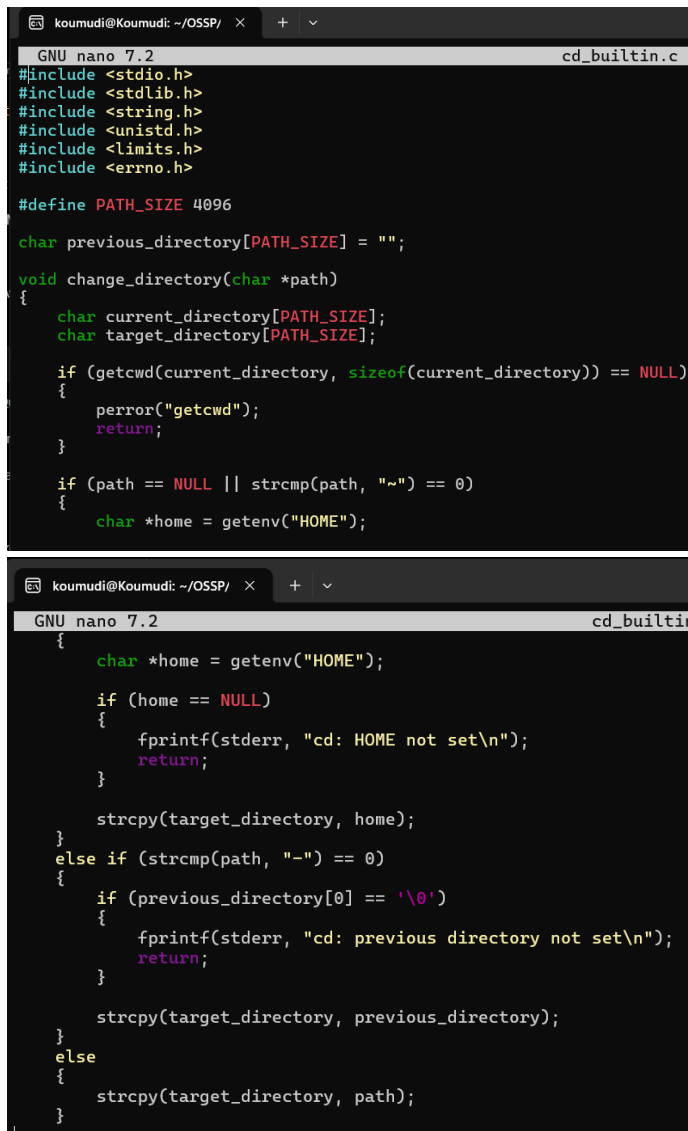
OSSP (Session 9&10)

Week 5

SKILL

Session 9:

Task 1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation



```
GNU nano 7.2 cd_builtin.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

#define PATH_SIZE 4096

char previous_directory[PATH_SIZE] = "";

void change_directory(char *path)
{
    char current_directory[PATH_SIZE];
    char target_directory[PATH_SIZE];

    if (getcwd(current_directory, sizeof(current_directory)) == NULL)
    {
        perror("getcwd");
        return;
    }

    if (path == NULL || strcmp(path, "~") == 0)
    {
        char *home = getenv("HOME");

        if (home == NULL)
        {
            fprintf(stderr, "cd: HOME not set\n");
            return;
        }

        strcpy(target_directory, home);
    }
    else if (strcmp(path, "-") == 0)
    {
        if (previous_directory[0] == '\0')
        {
            fprintf(stderr, "cd: previous directory not set\n");
            return;
        }

        strcpy(target_directory, previous_directory);
    }
    else
    {
        strcpy(target_directory, path);
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2 cd_builtin.c *
    }
    return;

    strcpy(previous_directory, current_directory);

    if (getcwd(current_directory, sizeof(current_directory)) != NULL)
    {
        printf("Current directory: %s\n", current_directory);
    }
}

int main()
{
    char input[PATH_SIZE];

    printf("CD Built-in Test\n");
    printf("Type a directory, '~', '-', or 'exit'\n");

    while (1)
    {
        printf("cd-test> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
```

```
        return;

        if (strncmp(input, "cd", 2) == 0)
        {
            char *path = input + 2;

            while (*path == ' ')
                path++;

            if (*path == '\0')
                path = NULL;

            change_directory(path);
        }
        else
        {
            printf("Unknown command. Use cd or exit.\n");
        }
    }

    return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + v
koumudi@Koumudi:~/OSSP/SKILL$ mkdir -p Session_9
cd Session_9
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ pwd
/home/koumudi/OSSP/SKILL/Session_9
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ nano cd_builtin.c
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ gcc cd_builtin.c -o cd_builtin
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ ./cd_builtin
CD Built-in Test
Type a directory, '~', '-', or 'exit'
cd-test> /
Unknown command. Use cd or exit.
cd-test> /tmp
Unknown command. Use cd or exit.
cd-test> -
Unknown command. Use cd or exit.
cd-test> /home/koumudi
Unknown command. Use cd or exit.
cd-test> ..
Unknown command. Use cd or exit.
cd-test> cd /
Current directory: /
cd-test> cd /tmp
Current directory: /tmp
cd-test> cd -
Current directory: /
cd-test> cd /home/koumudi
Current directory: /home/koumudi
cd-test> cd ..
Current directory: /home
cd-test> cd /does_not_exist
cd: /does_not_exist: No such file or directory
cd-test> exit
```

TASK 2: To Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

typedef int (*builtin_func)(char **args);

int builtin_cd(char **args);
int builtin_pwd(char **args);
int builtin_exit(char **args);
int builtin_help(char **args);

struct builtin
{
    char *name;
    builtin_func function;
};

struct builtin builtins[] =
{
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"exit", builtin_exit},
    {"help", builtin_help}
};

int builtin_count = sizeof(builtins) / sizeof(builtins[0]);

int builtin_cd(char **args)
{
    char *path;

    if (args[1] == NULL)
        path = getenv("HOME");
    else
        path = args[1];

    if (path == NULL)
```

```
koumudi@Koumudi: ~/OSSP/ × + ∨
GNU nano 7.2
    if (path == NULL)
    {
        printf("cd: HOME not set\n");
        return 1;
    }

    if (chdir(path) != 0)
    {
        perror("cd");
        return 1;
    }

    return 1;
}

int builtin_pwd(char **args)
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");

    return 1;
}

int builtin_exit(char **args)
{
    printf("Exiting shell...\n");
    return 0;
}

int builtin_help(char **args)
{
    printf("Built-in commands:\n");
    printf(" cd    Change directory\n");
    printf(" pwd    Display current directory\n");
    printf(" help    Display available commands\n");
    printf(" exit    Exit the shell\n");
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ∨
GNU nano 7.2
    printf(" exit    Exit the shell\n");
    return 1;
}

int execute_builtin(char **args)
{
    if (args[0] == NULL)
        return 1;

    for (int i = 0; i < builtin_count; i++)
    {
        if (strcmp(args[0], builtins[i].name) == 0)
        {
            return builtins[i].function(args);
        }
    }

    printf("Unknown command: %s\n", args[0]);
    return 1;
}

int main()
{
    char input[MAX_INPUT];

    printf("Mini Shell - Session 9\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        char *args[MAX_ARGS];
        int count = 0;
    }
```

```

printf("Mini Shell - Session 9\n");

while (1)
{
    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    char *args[MAX_ARGS];
    int count = 0;

    char *token = strtok(input, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (!execute_builtin(args))
        break;
}

return 0;
}

```

```

koumudi@Koumudi: ~/OSSP/ × + ▾
koumudi@Koumudi:~$ cd ~/OSSP/SKILL/Session_9
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ ls
cd_builtin cd_builtin.c dispatch_shell dispatch_shell.c
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ nano dispatch_shell.c
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ gcc dispatch_shell.c -o dispatch_shell
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ ./dispatch_shell
Mini Shell - Session 9
myshell> ./dispatch_shell
Unknown command: ./dispatch_shell
myshell> pwd
/home/koumudi/OSSP/SKILL/Session_9
myshell> help
Built-in commands:
  cd      - Change directory
  pwd     - Display current directory
  help    - Display help information
  exit    - Exit the shell
myshell> cd ..
myshell> pwd
/home/koumudi/OSSP/SKILL
myshell> exit
Exiting shell...
Shell terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_9$ |

```

SESSION 10:

TASK 1: To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behaviour.

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

#define INPUT_SIZE 1024

int main()
{
    char cwd[PATH_MAX];
    char input[INPUT_SIZE];
    int running = 1;

    printf("Mini Shell - Session 10 Task 1\n");

    while (running)
    {
        if (getcwd(cwd, sizeof(cwd)) != NULL)
        {
            printf("Current Directory: %s\n", cwd);
        }
        else
        {
            perror("getcwd");
        }
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2 she
    perror("getcwd");
}

printf("shell10> ");
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
{
    break;
}

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "pwd") == 0)
{
    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("Path: %s\n", cwd);
    }
    else
    {
        perror("pwd");
    }
}
else if (strcmp(input, "exit") == 0)
{
    ^G Help      ^O Write Out  ^W Where Is   ^K Cut
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste
```

```
koumudi@Koumudi: ~/OSSP/ × + ∨
GNU nano 7.2
        perror("pwd");
    }
}
else if (strcmp(input, "exit") == 0)
{
    printf("Exit request received.\n");
    printf("Saving shell state...\n");
    printf("Cleaning up resources...\n");

    running = 0;
}
else if (strlen(input) == 0)
{
    continue;
}
else
{
    printf("Unknown command: %s\n", input);
}
}

printf("Shell terminated successfully.\n");

return 0;
}
```

```
koumudi@Koumudi:~$ cd ~/OSSP/SKILL
koumudi@Koumudi:~/OSSP/SKILL$ ls
Session_1 Session_2 Session_3 Session_4 Session_5 Session_6 Session_7 Session_8 Session_9
koumudi@Koumudi:~/OSSP/SKILL$ mkdir Session_10
koumudi@Koumudi:~/OSSP/SKILL$ cd Session_10
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ nano shell_session10.c
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ gcc shell_session10.c -o shell_session10
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ ./shell_session10
Mini Shell - Session 10 Task 1
Current Directory: /home/koumudi/OSSP/SKILL/Session_10
shell10> ^C
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ gcc shell_session10.c -o shell_session10
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ ./shell_session10
Mini Shell - Session 10 Task 1
Current Directory: /home/koumudi/OSSP/SKILL/Session_10
shell10> pwd
Path: /home/koumudi/OSSP/SKILL/Session_10
Current Directory: /home/koumudi/OSSP/SKILL/Session_10
shell10> hello
Unknown command: hello
Current Directory: /home/koumudi/OSSP/SKILL/Session_10
shell10> exit
Exit request received.
Saving shell state...
Cleaning up resources...
Shell terminated successfully.
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ |
```

TASK 2: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define INPUT_SIZE 1024

int valid_variable_name(const char *name)
{
    if (name == NULL || name[0] == '\0')
    {
        return 0;
    }

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
    {
        return 0;
    }

    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
        {
            return 0;
        }
    }

    return 1;
}

void handle_export(char *command)
{
    char *argument = command + 7;

    while (*argument == ' ')
    {
        argument++;
    }

    if (*argument == '\0')
    {
        printf("Usage: export NAME=value\n");
        return;
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
}

char *equals = strchr(argument, '=');

if (equals == NULL)
{
    printf("Invalid syntax. Use: export NAME=value\n");
    return;
}

*equals = '\0';

char *name = argument;
char *value = equals + 1;

if (!valid_variable_name(name))
{
    printf("Invalid variable name: %s\n", name);
    return;
}

if (setenv(name, value, 1) != 0)
{
    perror("export");
    return;
}

printf("Environment variable updated successfully.\n");
printf("%s=%s\n", name, getenv(name));
}

void handle_printenv(char *command)
{
    char *name = command + 8;

    while (*name == ' ')
    {
        name++;
    }

    if (*name == '\0')
    {
        printf("Usage: printenv NAME\n");
        return;
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2

char *value = getenv(name);

if (value != NULL)
{
    printf("%s=%s\n", name, value);
}
else
{
    printf("Environment variable '%s' not found.\n", name);
}
}

int main()
{
    char input[INPUT_SIZE];
    int running = 1;

    printf("Mini Shell - Session 10 Task 2\n");

    while (running)
    {
        printf("export-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            continue;
        }

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            running = 0;
        }
        else if (strncmp(input, "export ", 7) == 0)
        {
            handle_export(input);
        }
    }
}
```

```
        else if (strncmp(input, "printenv ", 9) == 0)
        {
            handle_printenv(input);
        }
        else
        {
            printf("Unknown command: %s\n", input);
        }
    }

    printf("Shell terminated successfully.\n");

    return 0;
}
```

```
Shell terminated successfully.
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ nano export_shell.c
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ gcc export_shell.c -o export_shell
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ ./export_shell
Mini Shell - Session 10 Task 2
export-shell> export NAME=Koumudi
Environment variable updated successfully.
NAME=Koumudi
export-shell> printenv NAME
NAME=Koumudi
export-shell> export NAME=Student
Environment variable updated successfully.
NAME=Student
export-shell> export 123NAME=test
Invalid variable name: 123NAME
export-shell> export TEST
Invalid syntax. Use: export NAME=value
export-shell> exit
Exiting shell...
Shell terminated successfully.
koumudi@Koumudi:~/OSSP/SKILL/Session_10$ |
```